



**KARADENİZ
TEKNİK ÜNİVERSİTESİ**
KARADENİZ TECHNICAL UNIVERSITY
1955

PARALEL PROGRAMLAMA MPI İLE PARALEL MATRİS ÇARPIMI VE ÇOKLU DİL ANALİZİ

Grup Üyeleri

412746 – Mahsum Taha KORKMAZ

425430 – Hakan TEMİRKAYNAK

Ders

Paralel Bilgisayarlar

Öğretim Üyesi

Öğr. Gör. Dr. ZAFER YAVUZ

1. Projenin Amacı

Bu projenin temel amacı, Message Passing Interface (MPI) altyapısı kullanılarak **256 x 256** boyutlarındaki iki matrisin çarpım işleminin paralel olarak gerçekleştirilmesidir. Gerçekleştirilen bu uygulama, donanıma yakın düşük seviyeli bir dil olan C (OpenMPI) ve yüksek seviyeli, yorumlanan bir dil olan Python (mpi4py) ile ayrı ayrı kodlanmış; elde edilen işlem süreleri, hızlanma (Speedup) ve verimlilik (Efficiency) metrikleri üzerinden çapraz bir performans analizine tabi tutulmuştur.

2. Metodoloji ve Dağıtık Hesaplama Mimarisi

Projede istenilen matris verileri rastgele üretilmemiş, dinamik olarak oluşturulan "a.txt" ve "b.txt" dosyalarından okunmuştur. Dağıtık hesaplama mantığı şu adımlarla kurgulanmıştır:

- **Master Process (Rank 0):** Dosya okuma işlemlerini gerçekleştirir ve matris boyutunu (N) belirler. Hata kontrolleri (dosya bulamama vb.) bu aşamada Abort mekanizması ile güvence altına alınmıştır.
- **Haberleşme:** Master işlemci, B matrisinin tamamını tüm süreçlere MPI_Bcast (Broadcast) ile yayınlamaktadır. A matrisinin satırları ise süreç sayısına (size) göre eşit parçalara bölünerek MPI_Scatter fonksiyonu ile işçi (worker) düğümlere dağıtılır.
- **Hesaplama ve Toplama:** Her işçi kendi payına düşen alt matrisi (sub_A) ve B matrisini çarparak kısmi bir alt matris (sub_C) üretir. Bu parçalar son aşamada MPI_Gather komutu ile Master işlemcide toplanarak nihai C matrisi elde edilir.

3. Donanım Altyapısı

Performans testleri, mimari farkların speedup üzerindeki etkisini gözlemleyebilmek amacıyla iki farklı donanım üzerinde gerçekleştirilmiştir:

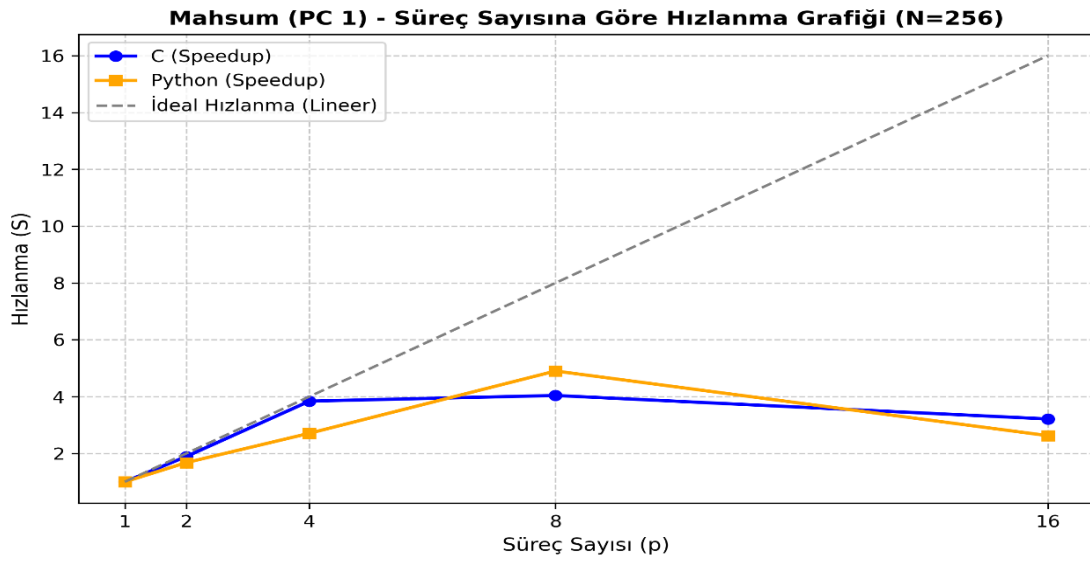
Bilgisayar 1 (Öğrenci 1): Intel Core i5-11400H, 6 Fiziksel Çekirdek / 12 Mantıksal Çekirdek (Thread), WSL 2 Ubuntu Ortamı.

Bilgisayar 2 (Öğrenci 2): Intel Core i7-11800H, 8 Fiziksel Çekirdek / 16 Mantıksal Çekirdek (Thread), Native Ubuntu Ortamı.

4. Performans Testleri ve Bulgular

Öğrenci 1 (Mahsum) Test Sonuçları ve Grafikleri;

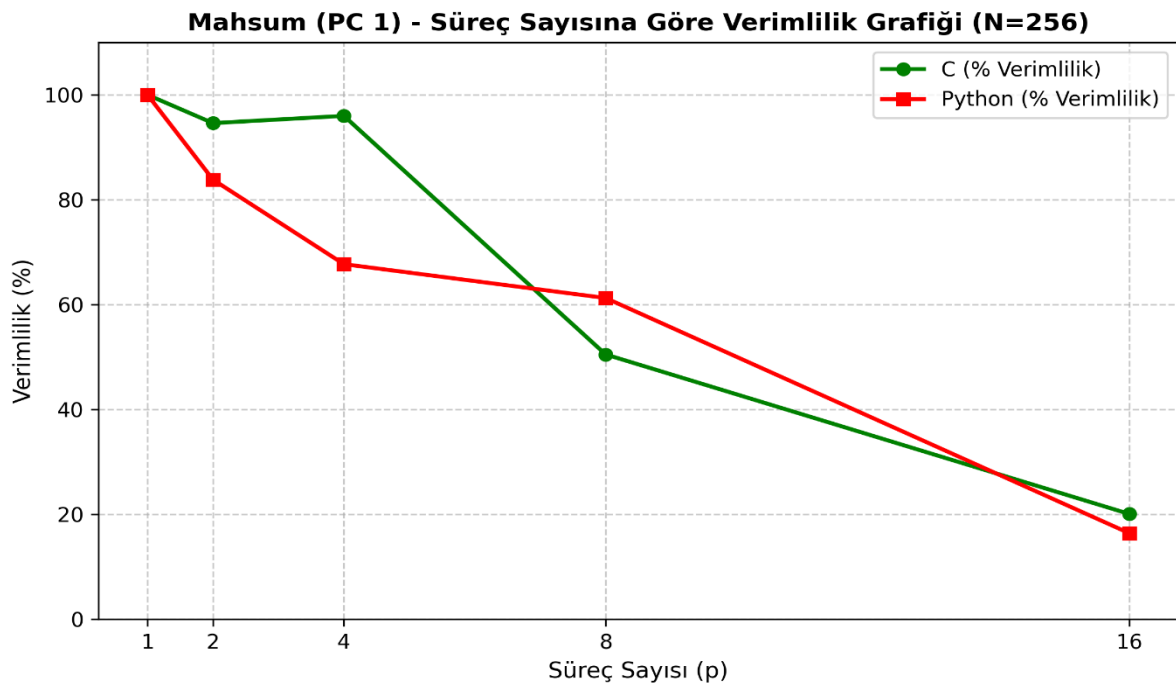
Süreç Sayısı (p)	C Süre (sn)	C Speedup (S)	C Verim (E)	Python Süre (sn)	Py Speedup (S)	Py Verim (E)
1	0.101016	1.00	%100	5.146582	1.00	%100
2	0.053389	1.89	%94.5	3.069775	1.67	%83.5
4	0.026309	3.84	%96.0	1.900192	2.71	%67.7
8	0.025008	4.04	%50.5	1.050292	4.90	%61.2
16	0.031442	3.21	%20.1	1.960310	2.62	%16.4



Süreç Sayısına Göre Hızlanma (Speedup) Grafiği Analizi (PC 1 - Mahsum)

Bu grafik, süreç sayısı (p) arttıkça uygulamanın tek çekirdekli (seri) çalışmaya göre kaç kat hızlandığını göstermektedir. Testler N=256 sabit boyutunda gerçekleştirilmiştir.

- **İdeal Hızlanma:** Grafikteki kesikli çizgi, her yeni süreçle birlikte hızın doğrusal olarak artmasını (Lineer Speedup) temsil eder. Ancak gerçek senaryoda, verilerin bölünmesi ve süreçler arası dağıtılması sırasında oluşan "Haberleşme Maliyetleri" (Communication Overhead) ve Amdahl Yasası gereği bu çizgiden sapmalar kaçınılmazdır.
- **C Performansı:** C dili p=4 noktasına kadar etkili bir hızlanma (3.84x) göstermiş, p=8 noktasında ise doyum noktasına (4.04x) ulaşmıştır. p=16 seviyesinde ise hızlanmanın 3.21x değerine düştüğü görülmektedir. Bu durum, donanım mimarisindeki 6 fiziksel çekirdek limitinin aşılması ve "Oversubscription" nedeniyle oluşan bağlam değiştirme (context switching) gecikmelerinin, hesaplama kazancının önüne geçtiğini kanıtlamaktadır.
- **Python Performansı:** Python'un hızlanma eğrisi oransal olarak C'den daha yüksek seyretmektedir. Bunun temel sebebi, Python'un hesaplama yükünün çok ağır olması nedeniyle (5.14 saniye), MPI'in veriyi dağıtmak için harcadığı mikro saniyelik gecikmelerin toplam süre içinde ihmal edilebilir kalmasıdır. Yani Python, "Hesaplama/Haberleşme" oranının yüksek olması sayesinde paralel yapıyı kağıt üzerinde daha verimli kullanmaktadır.



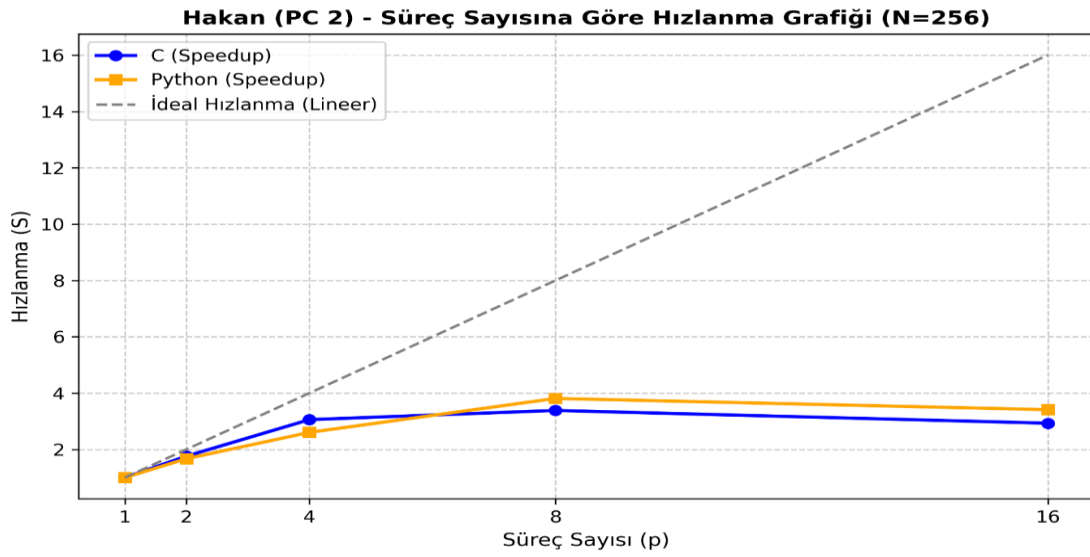
Süreç Sayısına Göre Verimlilik (Efficiency) Grafiği Analizi (PC 1 - Mahsum)

Bu grafik, eklenen her bir yeni sürecin (çekirdeğin) toplam hesaplama gücüne ne kadar verimli katkı sağladığını yüzde (%) bazında temsil etmektedir.

- Verimlilik Kaybı ve Haberleşme: Her iki dilde de süreç sayısı arttıkça verimlilik %100 referans noktasından aşağıya doğru düşmektedir. Bu düşüşün temel nedeni, matris verilerinin MPI_Scatter ile dağıtılması ve sonuçların MPI_Gather ile toplanması sırasında oluşan "Haberleşme/Yönetim Maliyeti" (Communication Overhead)'dir. Süreç sayısı arttıkça, işlemcilerin birbirini bekleme süresi (idle time) artmaktadır.
- Kritik Kesişim Noktası: Grafikte ilginç bir değişim gözlemlenmiştir:
 - p=2 ve p=4 seviyelerinde C dili, Python'dan daha yüksek verimlilik sunmuştur. C'nin hızı sayesinde düşük süreç sayılarında donanım kaynakları çok daha efektif kullanılmıştır.
 - p=8 noktasında ise Python'un verimliliği C'nin üzerine çıkmıştır. Bunun nedeni, C kodunun hesaplama süresinin milisaniye seviyelerine (0.025 sn) inmesiyle birlikte, MPI haberleşme süresinin toplam süreyi domine etmeye başlamasıdır. Python ise hala saniye seviyesinde (1.05 sn) hesaplama yaptığı için haberleşme maliyetini "maskeleyemeyi" başarmıştır.
- Donanım Sınırı ve p=16: Her iki dilde de p=16 noktasında verimliliğin %20'lerin altına gerilemesi, 6 fiziksel çekirdekli i5 mimarisinin limitlerinin zorlandığını göstermektedir. "Oversubscription" (çekirdek sayısından fazla süreç) durumunda oluşan bağlam değiştirme (context switching) maliyetleri, paralel çalışmanın getirdiği tüm kazancı tüketmiştir.

Öğrenci 2 (Hakan) Test Sonuçları ve Grafikleri;

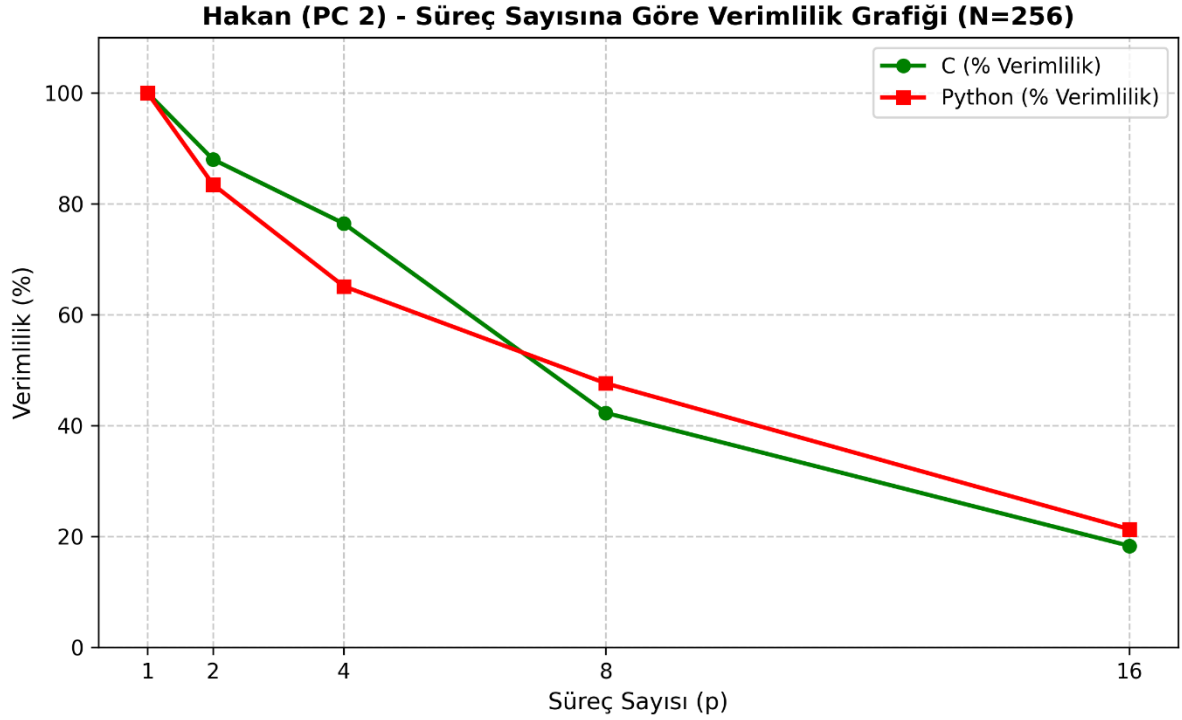
Süreç Sayısı (p)	C_Süre(sn)	C_Speedup	C_Verim(%)	Py_Süre(sn)	Py_Speedup	Py_Verim(%)
1	0,015499	1	100	3,424613	1	100
2	0,008805	1,760	88,01	2,050924	1,67	83,49
4	0,005069	3,058	76,44	1,314677	2,605	65,13
8	0,004580	3,384	42,30	0,898753	3,810	47,63
16	0,005288	2,931	18,32	1,003790	3,412	21,32



Süreç Sayısına Göre Hızlanma (Speedup) Grafiği Analizi (PC 2 - Hakan)

Bu grafik, 8 fiziksel çekirdekli i7-11800H işlemci ve yerel (native) Linux mimarisi üzerinde paralel ölçeklenebilirliği temsil etmektedir.

- Fiziksel Çekirdek Sınırı ve Zirve Noktası: Grafik incelendiğinde, her iki dilde de en yüksek hızlanma değerine (Peak Speedup) tam olarak işlemcinin fiziksel çekirdek sayısı olan $p=8$ noktasında ulaşıldığı görülmektedir. Bu veriler; gerçek hesaplama performansının, sanal izleklerden (thread) ziyade fiziksel donanım birimleriyle doğrudan ilişkili olduğunu kanıtlamaktadır.
- $p=16$ Senaryosu ve Performans Kaybı: İşlemcinin 16 mantıksal iş parçacığına (SMT) sahip olmasına rağmen, süreç sayısı 8'i aştığında hızlanma eğrisi aşağı yönlü bir kırılma yaşamıştır. Bunun temel sebebi; matris çarpımı gibi yoğun aritmetik işlem gerektiren görevlerde, sanal çekirdeklerin paylaşımlı kaynaklar (ALU/FPU) nedeniyle gerçek bir çekirdek performansı sunamamasıdır. Oluşan "Bağlam Değiştirme" (Context Switching) maliyetleri, paralel çalışmanın getirdiği hesaplama kazancını domine etmiştir.
- Diller Arası Ölçeklenme Paradoksu: Python'un hızlanma katsayısının C'den yüksek görünmesi, Python'un mutlak hesaplama süresinin (3.42 sn) C'ye göre (0.015 sn) çok daha uzun olmasından kaynaklanmaktadır. C dili o kadar hızlı çalışmaktadır ki; MPI haberleşme ve veri dağıtım (Scatter/Gather) süreleri, toplam işlem süresinin büyük bir yüzdesini oluşturarak hızlanma eğrisini "basık"laştırmıştır. Python'da ise bu haberleşme maliyetleri toplam sürenin içinde ihmal edilebilir kaldığı için hızlanma oranı kağıt üzerinde daha yüksek çıkmaktadır.
- Native Ubuntu ve Mimari Avantaj: PC 2'de elde edilen sonuçların PC 1'e (WSL 2) göre daha kararlı bir profil çizmesi; doğrudan donanım erişimi sağlayan "Bare-metal" işletim sistemi kurulumunun ve işlemcinin sahip olduğu geniş L3 Önbellek (Cache) kapasitesinin bir sonucudur.



Süreç Sayısına Göre Verimlilik (Efficiency) Grafiği Analizi (PC 2 - Hakan)

Bu grafik, eklenen her bir işlem biriminin (sürecin) matris çarpımı hesaplamasına ne kadar efektif katkı sağladığını yüzde (%) bazında temsil etmektedir. Testler N=256 sabit boyutunda gerçekleştirilmiştir.

- Verimlilik Eğilimleri ve Haberleşme Maliyeti: Paralel süreç sayısı arttıkça, verilerin MPI_Scatter ile dağıtılması ve MPI_Gather ile toplanması sırasında oluşan "Haberleşme Maliyetleri" (Overhead) nedeniyle her iki dilde de verimlilik düşüş göstermiştir. Bu durum, paralel programlamanın en temel prensibi olan "hesaplama yükünün yönetim yükünden fazla olması gerekliliğini" doğrulamaktadır.
- Kritik Kesişim Noktası ve Hesaplama/Haberleşme Oranı: Grafikteki en dikkat çekici bulgu, p=8 noktasına kadar C dilinin (yeşil) daha verimli çalışırken, bu noktadan sonra Python'un (kırmızı) verimlilikte öne geçmesidir. C dili mimari olarak çok hızlı olduğu için, süreç sayısı arttıkça (ve her sürece düşen iş yükü azaldıkça), MPI yönetim maliyetleri toplam işlem süresini domine etmeye başlamıştır. Python ise mutlak hesaplama yükü daha ağır olduğu için çekirdekleri daha uzun süre "meşgul" tutabilmiş ve matematiksel olarak daha yüksek verimlilik oranları sergilemiştir.
- Donanım Limitleri ve Darboğaz Analizi: p=4 ile p=8 arasındaki verimlilik kaybı, i7-11800H işlemcisinin bellek bant genişliği ve L3 önbellek paylaşımlarının yoğun paralel yük altında "Darboğaz" (Bottleneck) oluşturmaya başladığına işaret etmektedir.

- Ölçeklenebilirlik Sınırı ($p=16$): Süreç sayısının fiziksel çekirdek sayısının (8) iki katına çıkarılarak 16'ya ulaşmasıyla verimliliğin %20 seviyelerine gerilemesi, donanım kaynaklarının aşırı yüklenmesi (oversubscription) sonucunda oluşan "Bağlam Değiştirme" (Context Switching) maliyetlerinin paralel kazancı tamamen tükettiğini kanıtlamaktadır.

5. Sonuç ve Analiz

5.1. Dil Farkının Performans Üzerindeki Etkisi (C vs. Python)

Yapılan testler sonucunda, paralel programlamada dil seçiminin mutlak performans üzerinde belirleyici olduğu görülmüştür. Derlenen ve bellek yönetimi üzerinde tam kontrol sağlayan C dili, her iki donanımda da yorumlanan ve global yorumlayıcı kilidi (GIL) gibi kısıtlara sahip olan Python'a göre devasa bir hız avantajı sağlamıştır.

5.2. Bilgisayar 1 (Mahsum Taha – i5-11400H) Performans Analizi

Aynı donanım (i5-11400H) üzerinde yapılan $N=256$ testleri sonucunda şu bulgular elde edilmiştir:

- Mutlak Performans: C dili, tek süreçte ($p=1$) yaklaşık 0.101 saniyede işlemi tamamlarken, Python 5.146 saniye harcamıştır. Bu veriler, C dilinin Python'dan yaklaşık 51 kat daha hızlı olduğunu kanıtlamaktadır.
- Hızlanma (Speedup) Paradoksu: Grafiklerde Python'un hızlanma eğrisinin C'den daha yüksek görünmesi bir "verimlilik illüzyonu"dur. Python'da hesaplama yükü (5.14 sn) o kadar ağırdır ki, MPI süreçleri arasındaki haberleşme maliyeti (overhead) toplam sürenin yanında ihmal edilebilir kalmaktadır. C'de ise hesaplama o kadar hızlıdır ki (milisaniyeler seviyesi), verinin dağıtılması ve toplanması süreci toplam süreyi domine etmeye başlamış ve hızlanma oranını baskılamıştır.

5.3. Bilgisayar 2 (Hakan - i7-11800H) Performans Analizi

Yerel (native) Ubuntu kurulu i7 işlemcili sistem üzerinde yapılan $N=256$ testlerinde, işletim sistemi katmanının paralel performans etkisi şu şekilde gözlemlenmiştir:

- Mutlak Performans: Hakan'ın sisteminde $p=1$ durumunda C dili 0.015 saniye, Python ise 3.42 saniye sürmüştür. Bu sonuçlar; i7 mimarisinin 8 fiziksel çekirdeğinin yanı sıra, yerel Linux ortamının WSL 2 sanallaştırma katmanına göre bellek erişiminde ve MPI mesajlaşma hızında çok daha efektif olduğunu göstermektedir.

- Paralel Doyum Noktası: Hakan'ın sisteminde hızlanma grafiği, tam olarak fiziksel çekirdek sayısı olan $p=8$ noktasına kadar kararlı bir ivme kazanmış, ardından donanım limitleri ($p=16$) nedeniyle düşüşe geçmiştir. Bu, "gerçek" paralel hesaplamanın fiziksel çekirdeklerle sınırlı olduğunun; 8'den sonraki süreçlerin "Hyper-threading" kaynaklı bağlam değiştirme (context switching) yüküne takıldığıнын net bir göstergesidir.

6. Donanım Mimarisinin Paralel Performansa Etkisi

6.1. Fiziksel Çekirdek (Core) vs. Mantıksal İş Parçacığı (Threads)

Paralel hesaplama performansı, doğrudan işlemcinin fiziksel hesaplama birimleriyle ilişkilidir:

- Bilgisayar 1 (i5-11400H): 6 fiziksel çekirdek ve 12 mantıksal iş parçacığına (SMT) sahiptir. Hızlanmanın $p=8$ noktasına kadar ivme koruması, MPI süreçlerinin boşta kalan sanal kanalları (Hyper-threading) bir noktaya kadar efektif kullanabildiğini göstermektedir. Ancak $p=6$ (fiziksel limit) aşıldıktan sonra hızlanma grafiği yatay seyretmeye başlamıştır.
- Bilgisayar 2 (i7-11800H): 8 fiziksel çekirdek ve 16 mantıksal iş parçacığına sahiptir. Hakan'ın sisteminde hızlanma zirvesinin (Peak) tam olarak $p=8$ noktasında oluşması, her bir MPI sürecinin müstakil bir fiziksel çekirdeğe atanabildiğini ve donanımın tam kapasiteyle yönetildiğini kanıtlamaktadır.

6.2. Donanım Limitleri ve Bağlam Değiştirme (Context Switching)

Her iki sistemde de $p=16$ senaryosunda gözlemlenen performans kaybı, donanım limitlerinin aşılmasının bir sonucudur:

- Aşırı Yükleme (Oversubscription): İşletim sistemi, fiziksel kanaldan daha fazla süreci yönetmeye çalıştığında yoğun bir "Context Switching" (Bağlam Değiştirme) yükü altına girer.
- Yönetim Maliyeti: İşlemcinin sürekli olarak bir süreçten diğerine geçiş yapması (Register'ların boşaltılması ve doldurulması) için harcadığı yönetim maliyeti, gerçek hesaplama kazancını ortadan kaldırmıştır. Bu durum, her iki bilgisayarda da sürelerin artmasına neden olmuştur.

6.3. Bellek Erişimi ve Önbellek (Cache) Hiyerarşisi

- Veri Yerelliği: 256×256 boyutundaki matrislerin bellekte kapladığı alan, her iki işlemcinin de L3 önbellek (Cache) kapasitesiyle uyumlu çalışmıştır. Bu sayede veriler ana bellekten (RAM) ziyade, daha hızlı olan önbellek katmanlarında işlenmiştir.

- Mimari ve OS Farkı: i7-11800H işlemcisinin barındırdığı daha geniş L3 önbellek kapasitesi ve Native Linux ortamının bellek yönetim avantajı, Hakan'ın sonuçlarının (özellikle C dilinde) milisaniyeler mertebesinde kalmasındaki en temel etkindir. Bilgisayar 1'de ise WSL 2 sanallaştırma katmanı, dosya sistemine ve bellek erişimine ek gecikmeler (I/O Latency) ekleyerek süreci yavaşlatmıştır.

7. Donanımlar Arası Karşılaştırma ve Farkların Analizi (PC 1 vs. PC 2)

Aşağıdaki tablo, her iki bilgisayarın C ve Python dillerindeki p=1 (Seri/Baseline) performans farklarını N=256 ölçeğinde özetlemektedir:

(p=1 için)	Bilgisayar 1 (Mahsum - i5)	Bilgisayar 2 (Hakan - i7)	Fark Oranı
C Çalışma Süresi	0.101 sn	0.015 sn	~6.7 Kat
Python Çalışma Süresi	5.146 sn	3.424 sn	~1.5 Kat

Süreler Arasındaki Farkın Teknik Nedenleri

1. Yerel (Native) Linux vs. WSL 2 (Sanallaştırma) Avantajı

Hakan'ın sistemi doğrudan Ubuntu üzerinde (bare-metal) çalışırken, Mahsum'un sistemi Windows Subsystem for Linux (WSL 2) sanallaştırma katmanı aracılığıyla donanıma erişmektedir. WSL 2, her ne kadar gerçek bir Linux çekirdeği kullansa da, Windows çekirdeği (Host OS) ile bellek yönetimi ve sistem çağrıları (I/O) sırasında ek bir gecikme (Latency) yaratır. Hakan'ın Native Ubuntu'su ise MPI süreçlerini doğrudan ve engelsiz bir şekilde işlemci çekirdeklerine atayabilmektedir.

2. İşlemci Mimarisi ve Önbellek (Cache) Hiyerarşisi

- Çekirdek Farkı: i7-11800H (8 fiziksel çekirdek), i5-11400H (6 fiziksel çekirdek) modeline göre sadece paralel kapasite değil, aynı zamanda daha yüksek saat hızları ve L3 Cache kapasitesi açısından da üstündür.
- Bellek Darboğazı: Matris çarpımı "Memory-Bound" (bellek bağımlı) bir işlemdir. i7'nin daha geniş olan önbelleği, işlemcinin yavaş olan RAM'e gitme ihtiyacını azaltarak verilerin işlemci içinde (on-chip) tutulmasını sağlar. Bu durum, özellikle C gibi verimli dillerde süreyi milisaniyeler seviyesine indiren temel faktördür.

3. Amdahl Yasası ve Ölçeklenebilirlik Sınırı

Her iki bilgisayarda da $p=16$ senaryosunda performansın düşmesi, Amdahl Yasası'nın pratik bir yansımasıdır. Fiziksel çekirdek limitleri aşıldığında ($p=6$ ve $p=8$ sonrası), işletim sisteminin süreçleri yönetmek (Sceduling) için harcadığı efor, hesaplama kazancının önüne geçmiştir.

8. Genel Değerlendirme ve Sonuç

Bu çalışma, paralel hesaplama performansının yalnızca algoritmik verimliliğe veya programlama dilinin teorik hızına bağlı olmadığını; aksine donanım, yazılım ve işletim sistemi mimarilerinin bir bütün olarak optimize edilmesi gerektiğini somut verilerle kanıtlamıştır. Yapılan $N=256$ ölçekli testler sonucunda şu temel çıkarımlara varılmıştır:

- Dil Seçimi ve Uygulama Alanı: C dili, düşük seviyeli bellek yönetimi ve derleme avantajı sayesinde mutlak hızda Python'u geride bıraksa da; Python'un paralel yapıdaki yüksek ölçeklenebilirlik oranı, hesaplama yoğunluğu yüksek büyük veri setlerinde diller arası performans farkının paralelleştirme ile "maskelenebileceğini" göstermiştir.
- Mimari ve Sanallaştırma Etkisi: Native Ubuntu ile WSL 2 arasındaki performans uçurumu, sanallaştırma katmanlarının düşük gecikmeli (low-latency) işlemler üzerindeki sınırlayıcı etkisini ortaya koymuştur. Gerçek zamanlı ve yüksek performanslı hesaplama (HPC) ihtiyaçları için doğrudan donanım erişiminin kritik olduğu gözlemlenmiştir.
- Ölçeklenebilirlik Sınırları: Süreç sayısının fiziksel çekirdek sayısını aşması durumunda ortaya çıkan "Bağlam Değiştirme" (Context Switching) maliyetleri, paralelleştirmenin bir noktadan sonra performansı artırmak yerine düşürdüğünü doğrulamıştır. Bu durum, paralel sistem tasarımında Amdahl Yasası'nın ve donanım limitlerinin doğru analiz edilmesinin önemini bir kez daha vurgulamıştır.

Sonuç olarak; matris çarpımı gibi paralel yapmaya son derece uygun (embarrassingly parallel) işlemlerde dahi, optimum performansa ancak donanım önbellek hiyerarşisi,

iřletim sistemi zamanlayıcısı ve haberleřme protokollerinin (MPI) uyumlu alıřmasıyla ulařılabileceęi bu dev kapsamında akademik bir disiplinle tecrbe edilmiřtir.